CPSC 250
Philosophy of Complex Objects

Edward Brash

1 Motivation

Before we begin writing our own classes, it is useful to think about why classes exist in the first place.

A program often needs to represent a "thing" from the real world. That thing may not be naturally described by a single integer, float, string, list, tuple, or dictionary. Instead, it may require several related pieces of information, possibly of different data types.

For example, imagine that we want to store information about a student.

2 A Student as a Collection of Information

A student might have the following data associated with them:

Number	Information	Type
1	First name	string
2	Last name	string
3	Student ID	integer
4	Address	list
5	Grades	dictionary

The address itself might be a list:

```
address = [908, "Promenade Lane", "Williamsburg", "VA", 23185]
```

The grades might be a dictionary:

```
grades = {
    "HW": 94.2,
    "Quiz": 75.8,
    "Tests": 88.4,
    "Final": 74.2
}
```

Already, this is no longer a simple primitive value. It is a structured collection of information.

3 The List-Based Approach

One possible approach is to store all the student information in a list:

```
student_info = [first_name, last_name, student_id, address, grades]
```

Then we could access the homework grade by writing:

```
print(student_info[4]["HW"])
```

This works, but it requires us to remember that index 4 contains the grades dictionary.

4 Why This Works, But Is Not Ideal

The list-based approach technically works.
However, it has several disadvantages:

- The meaning of each index must be remembered.
- The code becomes difficult to read.
- The structure is fragile.
- If we change the order of the list, many parts of the program may break.

For example:

```
student_info[4]["HW"]
```

requires us to remember that:

- index 4 means "grades",
- and `"HW"` is a key inside the grades dictionary.

This can quickly become confusing.

5 A Complex Object

The key idea is that `student_info` is really a complex assembly of information.
It contains:

- strings,
- integers,
- lists,
- dictionaries,
- and possibly even other objects.

It is natural to call this entire collection an **object**.

6 Everything in Python Is Already an Object

In Python, almost everything can be thought of as an object:

- integers,
- floats,
- strings,
- booleans,
- lists,
- tuples,
- dictionaries,
- lists of lists,
- lists of dictionaries,
- dictionaries of lists,
- and so on.

Python already treats these things as objects internally.
The next step is to ask:

> Can we define our own object types?

The answer is yes.

7 Main Point

It would be useful to have a system that allows us to:

1. define objects in our own way,
2. decide what information belongs inside those objects,
3. define how users interact with those objects,
4. and protect the internal structure from accidental misuse.

This is the motivation for classes and object-oriented programming.

8 Classes and Object-Oriented Programming

A class allows us to define a new kind of object.

Instead of forcing student data into a list, we could imagine defining a `Student` class.

Then each student would be a `Student` object, with meaningful internal variables such as first name, last name, ID number, address, and grades.

The advantage is that the program becomes much more readable and maintainable.

9 Key Takeaways

- Real-world data often consists of many related pieces of information.
- Primitive types alone are not always enough.
- Lists and dictionaries can store complex information, but may become awkward.
- A complex assembly of information can be thought of as an object.
- Classes allow us to define our own object types.
- Object-oriented programming is about defining objects and controlling how users interact with them.